

TaskFlow - Session 4 Project Guide

Translating TaskFlow into TypeScript

In Session 1 we built the page with HTML and CSS.

In Session 2 we used JavaScript to show our own task data on the page.

In Session 3 we asked an API for the tasks instead of writing them ourselves.

For Session 3 homework, you built a lot more on top of that: 50 tasks instead of 15, a second API request for the people who own them, an owner's name under every task title, a "Tasks per person" summary, buttons that filter by person, an "All people" reset, and a message when nothing matches. All of that already existed, in JavaScript, before today.

In Session 4 we did not change what the app does at all. **The features were built in Session 3 homework. Today we are only adding types to code you already know.** We changed the **language** it is written in: `js/app.js` became `ts/app.ts`, and a small tool called `tsc` turns it back into the `js/app.js` the browser actually runs.

Keep this guide. You can read it at home to remember what we did.

Everything here comes from the real project files. Nothing is invented. This guide describes **Session 4 End** - the class translation - not the homework. Session 4 End still compiles with the same beginner settings used in class. The stricter settings, and most of the null-safety work they require, are homework, and this guide says so wherever it matters.

1. Where Did We Start Today?

Before today, TaskFlow already worked, completely, in JavaScript - including your Session 3 homework:

- The header, the three statistic cards, the filter buttons, the search box, and the task list.
- `let tasks = []`, filled by `loadTasks()` from the API - now 50 tasks, not 15.
- `let users = []`, filled by `loadUsers()` from a second API address.
- An owner's name under every task title, found by `getUserName(task.userId)`.
- A "Tasks per person" summary, built by `renderPeopleSummary()`, that only lists people who actually own a loaded task.
- Clickable person buttons, wired up by `addPersonListeners()`, and an "All people" button that clears the choice.
- `getVisibleTasks()` combining the status filter, the search, **and** the chosen person.

- `renderTasks()` , showing "No tasks to show." when all three checks together match nothing.
- `updateStats()` , `updateProgressText()` .
- A loading message and an error message, using `try / catch` - no `response.ok` . The people request has its **own** `try / catch` , so a failed people request never breaks the task list.

So the page was already alive, already correct, already handling a slow or failed request, and already handling two people's worth of extra JavaScript from your Session 3 homework. Today we changed **one thing**: the language the file is written in.

2. What Is TypeScript?

TypeScript is JavaScript, plus information about the kinds of values we expect.

Every JavaScript file you have already written is already valid TypeScript. TypeScript does not replace anything you know. It adds a way to say, directly in the code, "this is always a string" or "this function always gives back a boolean" - and then a tool checks that we kept that promise, **before the browser ever sees the code**.

TypeScript catches a mistake

|
v

You fix it on your own computer,
before anyone else ever sees it

That is the whole idea. Everything else in this guide is one consequence of it.

3. `.ts` and `.js` - Two Different Files Now

We write in `ts/app.ts` .

The browser still only ever loads `js/app.js` - exactly like every session before today.

Between the two sits a tool called `tsc` , the TypeScript compiler:

```
ts/app.ts  -->  tsc  -->  js/app.js  -->  browser
(we write this)  (checks types)  (browser runs this)
```

`tsc` reads `app.ts` , checks that every type promise we made is kept, and - if everything is fine

- writes out a plain JavaScript file. The browser has never heard of TypeScript. It only ever sees

plain, ordinary JavaScript.

Try it yourself. Open `js/app.js` after running `tsc`. Compare it to `ts/app.ts`. Every type annotation is gone. What is left is exactly the JavaScript you already know how to read.

4. `tsconfig.json`

This is the small file that tells `tsc` how to compile our project:

```
{
  "compilerOptions": {
    "target": "ES2017",
    "rootDir": "ts",
    "outDir": "js",

    "noImplicitAny": false,
    "strictNullChecks": false
  },
  "include": ["ts/**/*.ts"]
}
```

- `rootDir` / `outDir` - read our `.ts` files from `ts/`, write the compiled `.js` files into `js/`.
 - `target` - which version of JavaScript to produce. `ES2017` runs on every browser we care about.
 - `noImplicitAny` and `strictNullChecks` - both `false` for Session 4 End. These two settings are the entire subject of your homework. Turning them on, and fixing what they find, is CORE homework - not something we did in class.
-

5. Two Interfaces: `Task` and `User`

```
interface Task {
  userId: number;
  title: string;
  completed: boolean;
}

interface User {
```

```
    id: number;
    name: string;
}
```

An `interface` describes the shape of an object: which properties it has, and what type each one is.

Look at the real tasks API data:

```
{ "userId": 1, "id": 1, "title": "delectus aut autem", "completed": false }
```

Four properties arrive. Our `Task` interface only names three: `userId`, `title`, `completed`.

Why not `id`? Because nothing in `TaskFlow` reads `task.id` - not `renderTasks()`, not `updateStats()`, not `getVisibleTasks()`, not the summary. We *do* read `task.userId` now - to find the owner's name, and to filter by person - so it earns a place in the interface that it did not have in Session 3 class. `id` still does not. An interface only needs to describe what the code actually uses, not everything the API happens to send.

Look at the real users API data (<https://jsonplaceholder.typicode.com/users>):

```
{ "id": 1, "name": "Leanne Graham", "username": "Bret", "email":
  "Sincere@april.biz", "..." }
```

A person has many more fields than that - address, phone, company. `User` only names `id` and `name`, because those are the only two properties `getUserName()` and the summary ever read.

6. Typing the Two Arrays: `Task[]` and `User[]`

BEFORE:

```
let tasks = [];
let users = [];
```

AFTER:

```
let tasks: Task[] = [];
let users: User[] = [];
```

Same idea, twice. `Task[]` means "an array where every item is a `Task`." `User[]` means the same thing for people. Nothing about *how* either array is used changed - `for (const task of tasks)` and `for (const user of users)` still work exactly as before.

7. Primitive Types and Inference

```
let currentFilter: string = "all";
let searchText: string = "";
let selectedUserId: number = 0;
```

`string` and `number` are two of TypeScript's **primitive types** - the basic kinds of value, alongside `boolean`.

`selectedUserId` is still just a plain number - `0` means "nobody is chosen," any other value is a real person's `id`. Nothing new here: the same `number` type you have already used, describing a value whose *meaning* (not its type) happens to be "which person, if any."

Now look at `API_URL` and `USERS_URL`, which we did **not** change:

```
const API_URL = "https://jsonplaceholder.typicode.com/todos?_limit=50";
const USERS_URL = "https://jsonplaceholder.typicode.com/users";
```

No `: string` written on either, and none is needed. TypeScript looks at the text on the right of the `=` and works out the type by itself - this is called **inference**. We only write a type by hand when there is nothing to infer from (like `let tasks = []`, which could have held anything), or when writing it makes the code clearer for a reader.

8. Typing the DOM Lookups

```
const taskList = document.querySelector("#taskList");
const loadingMessage = document.querySelector("#loadingMessage");
const errorMessage = document.querySelector("#errorMessage");
const progressText = document.querySelector("#progressText");
const totalCount = document.querySelector("#totalCount");
const completedCount = document.querySelector("#completedCount");
const pendingCount = document.querySelector("#pendingCount");
const peopleList = document.querySelector("#peopleList");
const allPeopleButton = document.querySelector("#allPeopleButton");
const filterAllButton = document.querySelector("#filterAll");
const filterCompletedButton = document.querySelector("#filterCompleted");
const filterPendingButton = document.querySelector("#filterPending");
```

None of these lines changed - `peopleList` and `allPeopleButton` are new only in the sense that your Session 3 homework added them; nothing about typing them is different from any other

element here.

Be precise about what hovering actually shows today. Open `tsconfig.json` :

`strictNullChecks` is `false` . With that setting off, hover over any one of these in your editor and it shows plain `Element` - **not** `Element | null` . TypeScript is not tracking the possibility of `null` here at all, because we have told it not to.

`document.querySelector()` can still genuinely return `null` at runtime, though - that has nothing to do with `tsconfig.json` . It searches the page for something matching the id. If nothing matches - a typo in the HTML, an id that got renamed - there is nothing to return, so it returns `null` instead of an element. `strictNullChecks` does not change what the browser actually does; it only changes whether TypeScript makes us prove, in advance, that it won't happen.

Session 4 End does not protect most of these against being `null` . That is intentional - see Section 10. And it is exactly why tonight's homework matters: the moment `strictNullChecks` is turned on, every one of these hovers changes from `Element` to `Element | null` , and TypeScript starts refusing to compile until each one is checked first.

The one exception in kind, not in safety: `searchInput`

```
const searchInput = document.querySelector("#searchInput") as HTMLInputElement | null;
```

Later in the file we read `searchInput.value` . `.value` only exists on form elements like `<input>` - the plain `Element` type `querySelector()` normally returns does not have it.

as `HTMLInputElement | null` is a **type assertion**: we are telling TypeScript "trust me, treat this as *either* an `HTMLInputElement` or `null` ," rather than TypeScript working out the element kind by itself. Notice we kept `| null` in the assertion - we are only correcting *which kind* of element this is, not claiming it is guaranteed to exist. `searchInput` is still just as unprotected against `null` as every other constant in this section, and homework will ask you to check it too.

TypeScript does not verify the `HTMLInputElement` part of this assertion - it takes our word for it. Type assertions are a promise we make, not a check TypeScript performs.

9. Two API Requests, Two Type Assertions

```
tasks = await response.json() as Task[];
```

```
users = await response.json() as User[];
```

`response.json()` reads the text of an answer and turns it into JavaScript values - but by itself, TypeScript has no way to know their shape. Its type is `any`, which means "could be anything, no checking happens." Both lines above use the same idea, `as SomeType`, to tell TypeScript what shape to expect - one for the tasks request, one for the people request.

This is the third and fourth time in the file we use `as` to make a promise TypeScript cannot check by itself (after `searchInput`'s assertion). Every time, we are the ones responsible for getting it right.

10. The One Null-Guard Example: `renderTasks()`

```
function renderTasks(): void {
  if (!taskList) {
    return;
  }

  const visibleTasks: Task[] = getVisibleTasks();

  if (visibleTasks.length === 0) {
    taskList.innerHTML = `<li class="task-item">No tasks to show.</li>`;
    return;
  }

  let html: string = "";

  for (const task of visibleTasks) {
    // ...
  }

  taskList.innerHTML = html;
}
```

With `strictNullChecks` off, `taskList` is typed plain `Element` (Section 8) - TypeScript is not making us check it. We check anyway, on purpose, because `document.querySelector()` can still genuinely return `null` at runtime: is `taskList` actually there?

```
if (!taskList) { return; }
```

In plain English: "If `taskList` turned out not to exist, stop this function right now. There is nothing sensible to draw into a list that isn't there."

Notice the empty-state check (`visibleTasks.length === 0`, showing "No tasks to show.") sits **inside** the same guarded function, below the null check. It does not need its own separate guard - one check at the top protects everything below it, all the way to the end of the function.

This is the only place in the whole application where we guard against `null` today. Every other constant from Section 8 - `loadingMessage`, `errorMessage`, `progressText`, `totalCount`, `completedCount`, `pendingCount`, `peopleList`, `allPeopleButton`, `searchInput`, and the three filter buttons - is still completely unprotected. That is not something we forgot. It is the deliberate starting point for tonight's homework: turning on `strictNullChecks` finds every one of those, and the fix is this same idea - `if (!x) { ... }` - applied again. (It will not always look identical to this one; sometimes one check protects more than one constant, or more than one listener, at once. The idea stays the same.)

11. Typed Function Parameters and Return Values

BEFORE:

```
function getUsername(userId) {
```

AFTER:

```
function getUsername(userId: number): string {
```

```
    function getUsername(userId: number): string {
        for (const user of users) {
            if (user.id === userId) {
                return user.name;
            }
        }

        return "Unknown person";
    }
}
```

This is the session's dedicated example, because it does something no other function in this file does: it both **receives** a value (`userId: number`, a typed **parameter** - a promise about what kind of value this function will be given) and **gives one back** (`: string`, the **return type** - a promise about what kind of value it hands back).

Every other function in TaskFlow is one of two simpler shapes:

- **Receives values, returns nothing** - `: void`, for example `function setFilter(newFilter: string, clickedButton: Element): void`.
- **Receives nothing, returns something** - for example `function getVisibleTasks(): Task[]`.

`clickedButton` is explicitly written as `Element` - a promise this parameter will always hold a real element. That promise holds regardless of `strictNullChecks`: unlike the DOM constants from Section 8, `clickedButton`'s type was never derived from `document.querySelector()`, so there is no `null` possibility to track in the first place.

12. A Real Mistake TypeScript Caught

While translating `updateStats()`, this line stopped the whole file from compiling:

```
totalCount.textContent = tasks.length;
```

```
error TS2322: Type 'number' is not assignable to type 'string'.
```

`.textContent` only ever accepts a `string`. `tasks.length` is a `number`. In plain JavaScript, this line has always worked - JavaScript silently converts the number to text for you, every time. TypeScript refuses. It makes the conversion happen on purpose:

```
totalCount.textContent = String(tasks.length);
```

This was not a made-up example for the guide. It is a genuine bug TypeScript caught in our own real code while translating this session, and it is exactly the kind of thing "information about the kinds of values we expect" is for.

13. Async Functions Without a Written Return Type

```
async function loadTasks() {  
    showLoading();  
  
    try {  
        await loadUsers();  
    }  
}
```

```

    const response = await fetch(API_URL);
    tasks = await response.json() as Task[];

    hideLoading();
    updateStats();
    renderTasks();
    updateProgressText();
    renderPeopleSummary();
  } catch (error) {
    showError();
  }
}

```

Notice `loadTasks()` has no `: something` after its parentheses - and neither does `loadUsers()`.

`async function` always returns a `Promise`. TypeScript already knows this, and works out what kind of `Promise` by itself. Writing `Promise` types ourselves - saying exactly what is inside one - is a Session 8 topic. For now, leave `async` functions without a written return type and let TypeScript handle it.

`loadTasks()` `await s` `loadUsers()` first, so the people are already in `users` before a single task row is drawn - unchanged from your Session 3 homework, just now inside a typed function.

14. What Did Not Change

- `index.html` and `css/style.css` - not touched at all today.
- The order things happen in when the page opens - identical to Session 3 homework.
- Every function's actual logic - the three-way filter in `getVisibleTasks()`, the nested loop in `renderPeopleSummary()`, the owner lookup in `getUserName()`, the empty-state check in `renderTasks()` - all unchanged. Only type annotations were added around already-correct code.
- The look of the app, the two API addresses, the filters, the search box, the people summary, the person buttons, the progress sentence - all identical to Session 3 homework.

15. Common Session 4 Mistakes

1. Editing `js/app.js` directly

Why it's wrong: `js/app.js` is *generated*. Any change you make there is silently thrown away the next time `tsc` runs. **Check:** always edit `ts/app.ts`, then recompile.

2. Forgetting to recompile

Why it's wrong: the browser only ever loads `js/app.js`. If you save `app.ts` and refresh the browser without running `tsc` again, you are looking at old code. **Check:** run `tsc` after every change, and confirm it printed no errors before refreshing.

3. Writing `Promise<...>` on `loadTasks()` or `loadUsers()`

```
async function loadTasks(): Promise<void> {    // not this session
```

Why it's wrong: we have not learned `Promise` types yet - that is Session 8. Leave `async` functions without a written return type for now.

4. Guarding elements that were not asked for

Why it's wrong: it is not incorrect, exactly, but going beyond `renderTasks()`'s one guard during class or teamwork uses up the exact thing tonight's homework is meant to teach. **Check:** if you find yourself adding `if (!x) { return; }` somewhere other than `renderTasks()` before homework, that is a sign to stop and save it for tonight.

5. A number where text is expected

```
totalCount.textContent = tasks.length;    // error
```

Why it's wrong: `.textContent` only accepts `string`. `tasks.length` is a `number`. **Check:** error TS2322: Type 'number' is not assignable to type 'string'. Fix with `String(...)`.

6. Losing the `searchInput` assertion

```
const searchInput = document.querySelector("#searchInput"); // missing "as
HTMLInputElement | null"
```

Why it's wrong: without the assertion, `searchInput` is plain `Element`, which has no `.value` property. **Check:** `error TS2339: Property 'value' does not exist on type 'Element'.`

7. A missing or renamed `interface Task` OR `interface User`

Why it's wrong: every typed use of `Task / Task[]` or `User / User[]` depends on these two blocks existing with exactly these names. **Check:** `error TS2304: Cannot find name 'Task'.` (or `'User'`).

8. Adding a new guard inside `addPersonListeners()`

Why it's wrong: the `if (personButton)` check already inside this function is not new - it came from your Session 3 homework, for a plain JavaScript reason (somebody with no tasks has no button on the page). Typing this function only means adding `: void`. **Check:** if you added a second check here, ask whether the original Session 3 code already had one - it did.

16. A Simple Way to Debug Session 4

1. **Read the compiler's output first**, before touching the browser. `tsc` names the exact file and line number.
2. **Zero errors, but the app still looks wrong?** That is not a type bug - it is a logic bug. Debug it exactly the way you debugged Session 2 and Session 3: `console.log()`, DevTools Console, DevTools Network.
3. **Confused by an error message?** Read it slowly, out loud, one clause at a time. "Type 'X' is not assignable to type 'Y'" means: *you are trying to put an X-shaped value somewhere that promised to only ever hold a Y-shaped value.*
4. **The browser shows old behaviour after a fix?** Recompile, then hard-refresh.

17. Can You Explain This?

You should be able to say what each of these lines does, in your own words.

```
interface Task {
    userId: number;
    title: string;
    completed: boolean;
}
```

```
let tasks: Task[] = [];
```

```
const searchInput = document.querySelector("#searchInput") as HTMLInputElement
| null;
```

```
if (!taskList) {
    return;
}
```

```
function getUsername(userId: number): string {
```

If you can explain all five, you understand Session 4.

18. Quick Review Questions

1. What does TypeScript add that plain JavaScript does not have?
2. What does `tsc` actually do?
3. Does the browser ever run a `.ts` file directly?
4. Why does `Task` include `userId` this time, when the Session 3 class version did not?
5. Why doesn't `API_URL` need a written `: string`, but `let tasks` needs `: Task[]`?
6. With `strictNullChecks` off, what type does hovering over `taskList` actually show? What would it show if `strictNullChecks` were on, and why the difference?
7. Why does `searchInput` need `as HTMLInputElement | null`, rather than just `as HTMLInputElement`?
8. Where is the one place in this session's code that guards against a `null` element, and why there specifically?
9. What makes `getUsername()` different from every other function typed today?
10. Why did `totalCount.textContent = tasks.length;` fail to compile?
11. Why don't `loadTasks()` or `loadUsers()` have a written return type?

12. What settings are still `false` in `tsconfig.json` at the end of this session, and what will turning them on tonight actually do?

19. Answers

1. Information about the kinds of values a variable, parameter, or return value is expected to hold, checked before the code runs.
2. It reads `.ts` files, checks that every type promise was kept, and - if so - writes out a plain `.js` file the browser can run.
3. No. The browser only ever loads the compiled `.js` file.
4. Because `Task.userId` is now genuinely read by the app - to find the owner's name and to filter by person, both Session 3 homework features. An interface only needs to describe what the code actually uses, and `userId` is used now.
5. `API_URL` has a value on the right of `=` for TypeScript to infer from. `let tasks = []` does not - an empty array gives no hint about what it will hold, so we say so ourselves.
6. With `strictNullChecks` off, hovering shows plain `Element` - TypeScript is not tracking the possibility of `null` here. With `strictNullChecks` on, it would show `Element | null`, because `document.querySelector()` can genuinely return `null` at runtime if nothing on the page matches the id, and the stricter setting makes TypeScript account for that possibility in the type itself.
7. Because we read `searchInput.value` later, and `.value` only exists on `HTMLInputElement`, not on the plain `Element` type `querySelector()` normally returns - that is what the `HTMLInputElement` part fixes. The `| null` stays because `querySelector()` can still fail to find the element; the assertion only corrects *which kind* of element it would be, not whether it exists.
8. Inside `renderTasks()`, guarding `taskList`. It is the first place in the file where using a missing element would actually crash the page (`taskList.innerHTML = ...`), and it is the pattern teamwork and homework both reuse.
9. It is the first function that both receives a value (`userId`) and gives one back (a `name`). Every earlier function either takes values and returns nothing, or returns something without taking anything in.
10. `.textContent` only accepts a `string`. `tasks.length` is a `number`. TypeScript refuses to silently convert it the way JavaScript does - `String(tasks.length)` fixes it.
11. Because they are `async` functions, and TypeScript already knows those return a `Promise` - it works out the details by itself. Writing `Promise` types ourselves is a Session 8 topic.

12. `noImplicitAny` and `strictNullChecks`. In practice, almost everything you will see tonight comes from `strictNullChecks` - every function parameter was already typed in class, so `noImplicitAny` has little left to catch here. Turning `strictNullChecks` on will make TypeScript refuse to compile until every remaining possibly- `null` constant (everything except `taskList`, which is already guarded) is checked before use - the same idea shown in `renderTasks()`, applied again. This now includes `searchInput`, too: its assertion only fixed *which kind* of element it is, not whether it exists.

20. Glossary

TypeScript JavaScript, plus information about the kinds of values we expect.

.ts file A TypeScript source file. Not run directly by the browser.

tsc The TypeScript compiler. Reads `.ts` files, checks types, writes `.js` files.

tsconfig.json The file that tells `tsc` how to compile a project.

type annotation Writing a type by hand, such as `: string` or `: Task[]`.

inference TypeScript working out a type by itself, without an annotation, from the value on the right of `=`.

interface A named description of an object's shape - which properties it has, and what type each one is.

primitive type A basic kind of value: `string`, `number`, or `boolean`.

parameter type A promise about what kind of value a function will be given.

return type A promise about what kind of value a function gives back. `void` means "nothing."

type assertion (`as`) Telling TypeScript to treat a value as a specific type, without TypeScript verifying it. A promise we make, not a check TypeScript performs.

Element | null The type `document.querySelector()`'s result has once `strictNullChecks` is turned on: either the element, or `null` if nothing matched. With `strictNullChecks` off, as in Session 4 End, the same result is typed simply `Element` - TypeScript is not tracking the `null` possibility, even though it can still genuinely happen at runtime.

null guard A check, such as `if (!x) { return; }`, that stops a function before it uses a value that might be `null`.

noImplicitAny A stricter compiler setting that refuses to let a value silently be typed `any` (meaning "could be anything, no checking"). Off in Session 4 End. Homework CORE turns it on.

strictNullChecks A stricter compiler setting that requires every possibly- `null` value to be checked before use. Off in Session 4 End. Homework CORE turns it on.

You Have Translated TaskFlow Into TypeScript

TaskFlow behaves exactly like your Session 3 homework did. You can now:

- explain what TypeScript adds to JavaScript, in one sentence,
- read the `app.ts → tsc → app.js → browser` pipeline,
- write interfaces for two real object shapes,
- type variables, function parameters, and function return values,
- type a function that both receives and returns a value,
- use type assertions for two different DOM lookups and two different API responses,
- recognise `Element | null` and write a guard against it.

Look back at what happened across the four sessions:

- **Session 1** - HTML and CSS built what you see.
- **Session 2** - JavaScript took data and controlled what appears.
- **Session 3** - the data came from another computer, and your homework taught it about people.
- **Session 4** - the same code, now with types that catch mistakes before you run the page.

The most valuable thing you learned today is not any single keyword. It is this:

Every JavaScript file you already know how to write - including the homework you were proud of - is already valid TypeScript. What changed today is not the logic - it is how much the computer can check for us before we ever click refresh.

Tonight's homework is not a new topic. It is turning on the checking we deliberately left off today, and applying - everywhere it is needed - the one pattern you already saw work.